At Scale

🤘 Cache Me If You Can, Talk 3

Maciek Dragula Nathan Maurice

Tenstorrent

October 11, 2026

Limits of single chips

GPT-2 is a small model compared with today's models:

- For example, GLM-5.3 (753 B) and Kimi K3 (2.8 T) do not fit on one chip.

To run huge models, we distribute weights and calculations across many chips.

With **tensor parallelism**, each layer of the model is split across chips.

- Each chip computes its part, and the outputs are combined at each layer.

Inter-chip communication is important:

- The network is slower than memory and can be a new bandwidth limit.
- New limit: how much data is exchanged between chips per token.

But limits remain the same on large systems

Even with thousands of chips,

- Prefill stays compute-bound, and decode stays memory-bound.
- Text generation still needs a KV cache.
- Batching multiple tokens still improves throughput if capacity is available.
- Act IV optimizations such as fusion and trace remain important.

To hide network latency, we can overlap communication and calculations.

Running prefill and decode on the same device is not always efficient

Prefill and decode want different things.

- Decode needs memory bandwidth.
- Prefill needs compute.

Idea: We run prefill on one hardware partition and decode on another.

- We call this **disaggregation**: some devices run prefill, while others run decode.
- **The challenge**: managing and moving the KV cache between the two sides.

For single-card models, disaggregation is not needed.

But it is critical for models with more than 100B parameters that run on multiple-chip setups (e.g. Galaxy).

Large inference deployments must serve many users

Requests can arrive at *any* time. To handle multiple requests, we build batches at every step (**continuous batching**).

Each request has a matching KV cache. Total cache size can exceed memory size.

- To keep more cache, we can move some of it to system RAM or an SSD.
- To reduce wasted memory, we allocate the KV cache in **pages** as each request grows (like virtual memory).

An inference deployment schedules which requests run where and when, with 2 goals:

- Maximize cluster utilization (tokens/s).
- Keep work balanced across chips (TTFT).

Many tokens are easy to predict

Do we need a complex model to predict what comes after "1, 2, 3, 4"?

A small, fast model called a **draft model** can guess the next few tokens (e.g. *5, 6, 7*).

But we *still* have to verify that the predicted tokens are correct with the target model.

- This is similar to prefill: all guesses are checked in one pass.
- With a good token **acceptance rate**, it can be faster than running decode.

Correct guesses are kept, and generation continues at the first mispredicted token.

What we have seen today

1. Act I: The room's tokens per second versus the speed limit.
2. Act III: Users within the budget.
3. Act IV: Speedrun table, with the fastest versions rerun live on unseen texts.